memxlife

# Phase 3: Automated LLM Inference Runtime

Start date: May 26, 2026
First required submission: within 2 weeks after the start date
Maximum submissions: 2 submissions within 3 weeks after the start date

In this phase, you will build an agent that automatically generates an LLM inference runtime. The generated runtime must load a decoder-only model from the provided configuration and weights, maintain request state, and execute both prefill and decode efficiently. The runtime will be evaluated as a black box: we will compare its logits against a reference implementation for correctness, then drive it with serving-style request traces to measure throughput and memory behavior.

Correct inference is a hard requirement. A submission that does not pass correctness checking will not receive throughput credit.

## 1. Task

Your task is to implement an agent that generates an inference runtime for a small LLaMA-like decoder-only model. The generated runtime must support:

- loading model weights from a provided weight directory
- constructing runtime behavior from `model_config.json`
- prefill for prompt tokens
- decode for one new token per active request
- maintaining request state across calls
- removing finished requests
- returning logits that match the official reference implementation

You should design your runtime to work across different batch sizes, prompt lengths, decode lengths, and request orders. The official evaluation traces are not exposed in

advance.

## 2. What You Must Submit

Your submission must contain:

- `run.sh`
- your agent implementation and any files required by the agent

After `run.sh` finishes, your agent must generate:

- `workspace/engine.py`
- `workspace/results.log`

Do not treat `workspace/engine.py` as a manually submitted static solution. It is the output artifact produced by your agent. The log file is not used for scoring. It is provided so that you can inspect failures after submission, such as agent errors, code generation errors, compilation errors, or local self-test failures.

### Submission Contract

The evaluation system will enter the submission root directory and run:

```
bash run.sh
```

After `run.sh` finishes, the evaluation system will import:

```
workspace/engine.py
```

from the same directory and run the official correctness and throughput harness.

Your `run.sh` should invoke your agent. If the generated runtime needs custom extensions, generated files, or local self-tests, prepare them during this process. The evaluator will not use self-reported results from your log file; it will directly call the generated runtime.

## 3. Provided Inputs

The model configuration file is:

```
/target/model_config.json
```

In the public skeleton, the corresponding path is:

```
target/model_config.json
```

This file describes model structure, including hidden size, number of layers, number of attention heads, number of key-value heads, vocabulary size, and related parameters. Your runtime should not hardcode these values. It should construct the engine dynamically from the `model_config` argument passed to `create_engine(...)` .

The model weight directory is:

```
/target/weights
```

In the public skeleton, the weight file is:

```
target/weights/model.pt
```

The public skeleton uses a single PyTorch state dict. Hidden evaluation will provide weights through the same `weight_dir` argument.

## 4. Required Runtime Interface

`workspace/engine.py` must define:

```python
def create_engine(model_config: dict, weight_dir: str, device: str = "cuda"):
    return Engine(...)
```

The returned object must support:

```
class Engine:
    def prefill(self, request_ids, input_ids):
        ...

    def decode(self, request_ids, token_ids):
        ...

    def remove(self, request_ids):
        ...
```

## prefill(request_ids, input_ids)

Inputs:

- `request_ids` : a list of request IDs, such as `[0, 1, 2]`
- `input_ids` : a list of 1D `torch.Tensor` token sequences, one sequence per request

Output:

- a logits tensor with shape `[batch_size, vocab_size]`
- row `i` must contain the last-token logits for `request_ids[i]`

Calling `prefill(...)` for a request should create or replace that request's state. It should not clear the state of unrelated requests.

## decode(request_ids, token_ids)

Inputs:

- `request_ids` : a list of existing request IDs
- `token_ids` : a 1D `torch.Tensor` with shape `[batch_size]` , one new token per request

Output:

- a logits tensor with shape `[batch_size, vocab_size]`
- row `i` must contain the last-token logits after appending `token_ids[i]` to `request_ids[i]`

## remove(request_ids)

Input:

- `request_ids` : a list of finished request IDs

This method does not need to return anything. It should release or delete the request state associated with those IDs.

## 5. Correctness Checking

The official evaluator will use a PyTorch reference implementation with the same hidden model config and weights. We compare logits, not generated text.

Correctness is checked with:

$$|y_{\text{student}} - y_{\text{ref}}| \leq \text{atol} + \text{rtol} \cdot |y_{\text{ref}}|$$

The public skeleton uses:

$$\text{atol} = 10^{-2}, \quad \text{rtol} = 10^{-2}$$

The public correctness test uses:

```
torch.allclose(student_logits, ref_logits, atol=1e-2, rtol=1e-2)
```

Correctness tests cover:

- single-request prefill
- single-request decode
- multi-request prefill
- multi-request decode
- inserting new requests
- removing requests and continuing to decode other requests

If a case fails correctness, that case receives no throughput credit.

## 6. Throughput Evaluation

The official evaluator will drive your engine directly:

```
engine = create_engine(model_config, weight_dir, device)
engine.prefill(...)
engine.decode(...)
engine.remove(...)
```

The measured region includes calls to:

- `prefill(...)`
- `decode(...)`
- `remove(...)`

The measured region does not include `create_engine(...)` or initial weight loading. If you perform lazy compilation or expensive initialization inside the measured calls, that time will be counted.

Throughput is reported as:

$$\text{tokens/s} = \frac{\text{prefill tokens} + \text{decode tokens}}{\text{elapsed seconds}}$$

Decode throughput is reported as:

$$\text{decode tokens/s} = \frac{\text{decode tokens}}{\text{elapsed seconds}}$$

The public benchmark includes three case families:

- `prefill` : batched long-prompt prefill
- `decode` : multiple active requests with repeated decode steps
- `mixed` : a serving-style trace with prefill, decode, and remove operations

Hidden evaluation will use the same interface and evaluation style, but with hidden model sizes, weights, batch sizes, prompt lengths, decode steps, and request traces.

## 7. Scoring Strategy

Correctness is a hard requirement.

A submission that does not pass correctness checking will not receive throughput credit.

For submissions that pass correctness, the final score is:

- 70% Throughput
- 30% Agent Implementation / Engineering Methodology

### Throughput

Throughput scoring is based on official benchmark traces. The evaluator will use warmup, repeated measurements, and median timing where appropriate.

The benchmark will consider prefill, decode, and mixed serving behavior. You should optimize for the overall runtime behavior of the engine, not only for one isolated call pattern.

### Agent Implementation / Engineering Methodology

This part rewards submissions that show a real engineering workflow, including factors such as:

- clear runtime organization
- local correctness tests against a reference
- benchmarking and profiling for decision making
- iterative improvement
- robust handling of different model configs and request patterns
- reproducibility through `run.sh` and logs

The project is not asking for a hand-written static solution that only works for the public toy case. A strong submission should use the public inputs to validate the interface, then build a runtime that generalizes to hidden cases.

## 8. Allowed Optimization Directions

You may optimize the runtime using techniques such as:

- real per-layer KV cache
- batched prefill and decode
- PyTorch SDPA or other PyTorch primitives

- Triton kernels
- C++/CUDA extensions
- custom kernels for RMSNorm, RoPE, attention, MLP, or cache operations
- better memory layout and request-state management

You should avoid relying on complete inference frameworks as the final runtime implementation. The evaluator expects your `engine.py` to implement the required interface directly.

## 9. Public Skeleton

If the public weight file is missing, regenerate it with:

```
python3 scripts/generate_toy_weights.py \
  --config target/model_config.json \
  --output target/weights/model.pt
```

Run the public correctness test:

```
python3 evaluator/test_correctness.py \
  --engine workspace/engine.py \
  --model-config target/model_config.json \
  --weight-dir target/weights \
  --device auto
```

Run the public throughput benchmark:

```
python3 evaluator/benchmark_throughput.py \
  --engine workspace/engine.py \
  --model-config target/model_config.json \
  --weight-dir target/weights \
  --device auto
```

Or run both:

```
bash scripts/run_public_tests.sh
```

If your default `python3` does not have PyTorch, specify a Python interpreter:

```
PYTHON=/path/to/python-with-torch bash scripts/run_public_tests.sh
```

## 10. Baseline

The public skeleton already contains a sample generated artifact at `workspace/engine.py` so that you can run the evaluator immediately. This file is a minimal PyTorch baseline. It stores the full token sequence for each request and recomputes the full sequence on every decode call. This is slow, but it demonstrates the required interface and correct request semantics. In your own submission, your agent must generate `workspace/engine.py` after `run.sh` starts.

Important optimization directions include:

- implement a real per-layer KV cache
- make `decode(...)` compute only the new token
- batch work across requests
- reduce Python overhead
- optimize attention, MLP, RMSNorm, RoPE, and cache operations
- adapt implementation choices to `model_config.json`

## 11. Summary

In this project, you are building an agent that automatically generates an inference runtime for a decoder-only language model.

Your submission should:

- provide `run.sh`
- invoke your agent from `run.sh`
- generate `workspace/engine.py`
- implement `create_engine(...)`
- support `prefill(...)`, `decode(...)`, and `remove(...)`

- maintain independent request state using request IDs
- match reference logits
- optimize throughput on serving-style traces

Only correct implementations receive throughput credit. Among correct submissions, the score is based on:

- 70% throughput
- 30% agent implementation / engineering methodology